

$$\begin{aligned}
&\leq \sum_{i=1}^{\infty} \alpha^{i-1} + 0 \\
&= \sum_{i=0}^{\infty} \alpha^i \\
&= \frac{1}{1-\alpha} \quad (\text{by equation (A.7) on page 1142 because } 0 \leq \alpha < 1) . \quad \blacksquare
\end{aligned}$$

If α is a constant, Theorem 11.6 predicts that an unsuccessful search runs in $O(1)$ time. For example, if the hash table is half full, the average number of probes in an unsuccessful search is at most $1/(1 - .5) = 2$. If it is 90% full, the average number of probes is at most $1/(1 - .9) = 10$.

Theorem 11.6 yields almost immediately how well the HASH-INSERT procedure performs.

Corollary 11.7

Inserting an element into an open-address hash table with load factor α , where $\alpha < 1$, requires at most $1/(1 - \alpha)$ probes on average, assuming independent uniform permutation hashing and no deletions.

Proof An element is inserted only if there is room in the table, and thus $\alpha < 1$. Inserting a key requires an unsuccessful search followed by placing the key into the first empty slot found. Thus, the expected number of probes is at most $1/(1 - \alpha)$. ■

It takes a little more work to compute the expected number of probes for a successful search.

Theorem 11.8

Given an open-address hash table with load factor $\alpha < 1$, the expected number of probes in a successful search is at most

$$\frac{1}{\alpha} \ln \frac{1}{1-\alpha} ,$$

assuming independent uniform permutation hashing with no deletions and assuming that each key in the table is equally likely to be searched for.

Proof A search for a key k reproduces the same probe sequence as when the element with key k was inserted. If k was the $(i + 1)$ st key inserted into the hash table, then the load factor at the time it was inserted was i/m , and so by Corollary 11.7, the expected number of probes made in a search for k is at most $1/(1 - i/m) = m/(m - i)$. Averaging over all n keys in the hash table gives us

the expected number of probes in a successful search:

$$\begin{aligned}
 \frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} &= \frac{m}{n} \sum_{i=0}^{n-1} \frac{1}{m-i} \\
 &= \frac{1}{\alpha} \sum_{k=m-n+1}^m \frac{1}{k} \\
 &\leq \frac{1}{\alpha} \int_{m-n}^m \frac{1}{x} dx \quad (\text{by inequality (A.19) on page 1150}) \\
 &= \frac{1}{\alpha} (\ln m - \ln(m-n)) \\
 &= \frac{1}{\alpha} \ln \frac{m}{m-n} \\
 &= \frac{1}{\alpha} \ln \frac{1}{1-\alpha}. \quad \blacksquare
 \end{aligned}$$

If the hash table is half full, the expected number of probes in a successful search is less than 1.387. If the hash table is 90% full, the expected number of probes is less than 2.559. If $\alpha = 1$, then in an unsuccessful search, all m slots must be probed. Exercise 11.4-4 asks you to analyze a successful search when $\alpha = 1$.

Exercises

11.4-1

Consider inserting the keys 10, 22, 31, 4, 15, 28, 17, 88, 59 into a hash table of length $m = 11$ using open addressing. Illustrate the result of inserting these keys using linear probing with $h(k, i) = (k + i) \bmod m$ and using double hashing with $h_1(k) = k$ and $h_2(k) = 1 + (k \bmod (m - 1))$.

11.4-2

Write pseudocode for HASH-DELETE that fills the deleted key's slot with the special value DELETED, and modify HASH-SEARCH and HASH-INSERT as needed to handle DELETED.

11.4-3

Consider an open-address hash table with independent uniform permutation hashing and no deletions. Give upper bounds on the expected number of probes in an unsuccessful search and on the expected number of probes in a successful search when the load factor is $3/4$ and when it is $7/8$.

11.4-4

Show that the expected number of probes required for a successful search when $\alpha = 1$ (that is, when $n = m$), is H_m , the m th harmonic number.

★ 11.4-5

Show that, with double hashing, if m and $h_2(k)$ have greatest common divisor $d \geq 1$ for some key k , then an unsuccessful search for key k examines $(1/d)$ th of the hash table before returning to slot $h_1(k)$. Thus, when $d = 1$, so that m and $h_2(k)$ are relatively prime, the search may examine the entire hash table. (*Hint:* See Chapter 31.)

★ 11.4-6

Consider an open-address hash table with a load factor α . Approximate the nonzero value α for which the expected number of probes in an unsuccessful search equals twice the expected number of probes in a successful search. Use the upper bounds given by Theorems 11.6 and 11.8 for these expected numbers of probes.

11.5 Practical considerations

Efficient hash table algorithms are not only of theoretical interest, but also of immense practical importance. Constant factors can matter. For this reason, this section discusses two aspects of modern CPUs that are not included in the standard RAM model presented in Section 2.2:

Memory hierarchies: The memory of modern CPUs has a number of levels, from the fast registers, through one or more levels of *cache memory*, to the main-memory level. Each successive level stores more data than the previous level, but access is slower. As a consequence, a complex computation (such as a complicated hash function) that works entirely within the fast registers can take less time than a single read operation from main memory. Furthermore, cache memory is organized in *cache blocks* of (say) 64 bytes each, which are always fetched together from main memory. There is a substantial benefit for ensuring that memory usage is local: reusing the same cache block is much more efficient than fetching a different cache block from main memory.

The standard RAM model measures efficiency of a hash-table operation by counting the number of hash-table slots probed. In practice, this metric is only a crude approximation to the truth, since once a cache block is in the cache, successive probes to that cache block are much faster than probes that must access main memory.

Advanced instruction sets: Modern CPUs may have sophisticated instruction sets that implement advanced primitives useful for encryption or other forms of cryptography. These instructions may be useful in the design of exceptionally efficient hash functions.

Section 11.5.1 discusses linear probing, which becomes the collision-resolution method of choice in the presence of a memory hierarchy. Section 11.5.2 suggests how to construct “advanced” hash functions based on cryptographic primitives, suitable for use on computers with hierarchical memory models.

11.5.1 Linear probing

Linear probing is often disparaged because of its poor performance in the standard RAM model. But linear probing excels for hierarchical memory models, because successive probes are usually to the same cache block of memory.

Deletion with linear probing

Another reason why linear probing is often not used in practice is that deletion seems complicated or impossible without using the special DELETED value. Yet we’ll now see that deletion from a hash table based on linear probing is not all that difficult, even without the DELETED marker. The deletion procedure works for linear probing, but not for open-address probing in general, because with linear probing keys all follow the same simple cyclic probing sequence (albeit with different starting points).

The deletion procedure relies on an “inverse” function to the linear-probing hash function $h(k, i) = (h_1(k) + i) \bmod m$, which maps a key k and a probe number i to a slot number in the hash table. The inverse function g maps a key k and a slot number q , where $0 \leq q < m$, to the probe number that reaches slot q :

$$g(k, q) = (q - h_1(k)) \bmod m.$$

If $h(k, i) = q$, then $g(k, q) = i$, and so $h(k, g(k, q)) = q$.

The procedure LINEAR-PROBING-HASH-DELETE on the facing page deletes the key stored in position q from hash table T . Figure 11.6 shows how it works. The procedure first deletes the key in position q by setting $T[q]$ to NIL in line 2. It then searches for a slot q' (if any) that contains a key that should be moved to the slot q just vacated by key k . Line 9 asks the critical question: does the key k' in slot q' need to be moved to the vacated slot q in order to preserve the accessibility of k' ? If $g(k', q) < g(k', q')$, then during the insertion of k' into the table, slot q was examined but found to be already occupied. But now slot q , where a search will look for k' , is empty. In this case, key k' moves to slot q in line 10, and the

0		0	
1		1	
2	82	2	82
3	43	3	93
4	74	4	74
5	93	5	92
6	92	6	
7		7	
8	18	8	18
9	38	9	38
(a)		(b)	

Figure 11.6 Deletion in a hash table that uses linear probing. The hash table has size 10 with $h_1(k) = k \bmod 10$. **(a)** The hash table after inserting keys in the order 74, 43, 93, 18, 82, 38, 92. **(b)** The hash table after deleting the key 43 from slot 3. Key 93 moves up to slot 3 to keep it accessible, and then key 92 moves up to slot 5 just vacated by key 93. No other keys need to be moved.

search continues, to see whether any later key also needs to be moved to the slot q' that was just freed up when k' moved.

LINEAR-PROBING-HASH-DELETE(T, q)

```

1  while TRUE
2       $T[q] = \text{NIL}$                                 // make slot  $q$  empty
3       $q' = q$                                        // starting point for search
4      repeat
5           $q' = (q' + 1) \bmod m$                     // next slot number with linear probing
6           $k' = T[q']$                                // next key to try to move
7          if  $k' == \text{NIL}$ 
8              return                               // return when an empty slot is found
9          until  $g(k', q) < g(k', q')$               // was empty slot  $q$  probed before  $q'$ ?
10          $T[q] = k'$                                // move  $k'$  into slot  $q$ 
11          $q = q'$                                    // free up slot  $q'$ 
```

Analysis of linear probing

Linear probing is popular to implement, but it exhibits a phenomenon known as *primary clustering*. Long runs of occupied slots build up, increasing the average

search time. Clusters arise because an empty slot preceded by i full slots gets filled next with probability $(i + 1)/m$. Long runs of occupied slots tend to get longer, and the average search time increases.

In the standard RAM model, primary clustering is a problem, and general double hashing usually performs better than linear probing. By contrast, in a hierarchical memory model, primary clustering is a beneficial property, as elements are often stored together in the same cache block. Searching proceeds through one cache block before advancing to search the next cache block. With linear probing, the running time for a key k of HASH-INSERT, HASH-SEARCH, or LINEAR-PROBING-HASH-DELETE is at most proportional to the distance from $h_1(k)$ to the next empty slot.

The following theorem is due to Pagh et al. [351]. A more recent proof is given by Thorup [438]. We omit the proof here. The need for 5-independence is by no means obvious; see the cited proofs.

Theorem 11.9

If h_1 is 5-independent and $\alpha \leq 2/3$, then it takes expected constant time to search for, insert, or delete a key in a hash table using linear probing. ■

(Indeed, the expected operation time is $O(1/\epsilon^2)$ for $\alpha = 1 - \epsilon$.)

★ **11.5.2 Hash functions for hierarchical memory models**

This section illustrates an approach for designing efficient hash tables in a modern computer system having a memory hierarchy.

Because of the memory hierarchy, linear probing is a good choice for resolving collisions, as probe sequences are sequential and tend to stay within cache blocks. But linear probing is most efficient when the hash function is complex (for example, 5-independent as in Theorem 11.9). Fortunately, having a memory hierarchy means that complex hash functions can be implemented efficiently.

As noted in Section 11.3.5, one approach is to use a cryptographic hash function such as SHA-256. Such functions are complex and sufficiently random for hash table applications. On machines with specialized instructions, cryptographic functions can be quite efficient.

Instead, we present here a simple hash function based only on addition, multiplication, and swapping the halves of a word. This function can be implemented entirely within the fast registers, and on a machine with a memory hierarchy, its latency is small compared with the time taken to access a random slot of the hash table. It is related to the RC6 encryption algorithm and can for practical purposes be considered a “random oracle.”

The wee hash function

Let w denote the word size of the machine (e.g., $w = 64$), assumed to be even, and let a and b be w -bit unsigned (nonnegative) integers such that a is odd. Let $\text{swap}(x)$ denote the w -bit result of swapping the two $w/2$ -bit halves of w -bit input x . That is,

$$\text{swap}(x) = (x \ggg (w/2)) + (x \lll (w/2))$$

where “ $\ggg$ ” is “logical right shift” (as in equation (11.2)) and “ $\lll$ ” is “left shift.” Define

$$f_a(k) = \text{swap}((2k^2 + ak) \bmod 2^w).$$

Thus, to compute $f_a(k)$, evaluate the quadratic function $2k^2 + ak$ modulo 2^w and then swap the left and right halves of the result.

Let r denote a desired number of “rounds” for the computation of the hash function. We’ll use $r = 4$, but the hash function is well defined for any nonnegative r . Denote by $f_a^{(r)}(k)$ the result of iterating f_a a total of r times (that is, r rounds) starting with input value k . For any odd a and any $r \geq 0$, the function $f_a^{(r)}$, although complicated, is one-to-one (see Exercise 11.5-1). A cryptographer would view $f_a^{(r)}$ as a simple block cipher operating on w -bit input blocks, with r rounds and key a .

We first define the wee hash function h for short inputs, where by “short” we means “whose length t is at most w -bits,” so that the input fits within one computer word. We would like inputs of different lengths to be hashed differently. The *wee hash function* $h_{a,b,t,r}(k)$ for parameters a , b , and r on t -bit input k is defined as

$$h_{a,b,t,r}(k) = (f_{a+2t}^{(r)}(k + b)) \bmod m. \quad (11.7)$$

That is, the hash value for t -bit input k is obtained by applying $f_{a+2t}^{(r)}$ to $k + b$, then taking the final result modulo m . Adding the value b provides hash-dependent randomization of the input, in a way that ensures that for variable-length inputs the 0-length input does not have a fixed hash value. Adding the value $2t$ to a ensures that the hash function acts differently for inputs of different lengths. (We use $2t$ rather than t to ensure that the key $a + 2t$ is odd if a is odd.) We call this hash function “wee” because it uses a tiny amount of memory—more precisely, it can be implemented efficiently using only the computer’s fast registers. (This hash function does not have a name in the literature; it is a variant we developed for this textbook.)

Speed of the wee hash function

It is surprising how much efficiency can be bought with locality. Experiments (unpublished, by the authors) suggest that evaluating the wee hash function takes less

time than probing a *single* randomly chosen slot in a hash table. These experiments were run on a laptop (2019 MacBook Pro) with $w = 64$ and $a = 123$. For large hash tables, evaluating the wee hash function was 2 to 10 times faster than performing a single probe of the hash table.

The wee hash function for variable-length inputs

Sometimes inputs are long—more than one w -bit word in length—or have variable length, as discussed in Section 11.3.5. We can extend the wee hash function, defined above for inputs that are at most single w -bit word in length, to handle long or variable-length inputs. Here is one method for doing so.

Suppose that an input k has length t (measured in bits). Break k into a sequence $\langle k_1, k_2, \dots, k_u \rangle$ of w -bit words, where $u = \lceil t/w \rceil$, k_1 contains the least-significant w bits of k , and k_u contains the most significant bits. If t is not a multiple of w , then k_u contains fewer than w bits, in which case, pad out the unused high-order bits of k_u with 0-bits. Define the function *chop* to return a sequence of the w -bit words in k :

$$\text{chop}(k) = \langle k_1, k_2, \dots, k_u \rangle.$$

The most important property of the chop operation is that it is one-to-one, given t : for any two t -bit keys k and k' , if $k \neq k'$ then $\text{chop}(k) \neq \text{chop}(k')$, and k can be derived from $\text{chop}(k)$ and t . The chop operation also has the useful property that a single-word input key yields a single-word output sequence: $\text{chop}(k) = \langle k \rangle$.

With the chop function in hand, we specify the wee hash function $h_{a,b,t,r}(k)$ for an input k of length t bits as follows:

$$h_{a,b,t,r}(k) = \text{WEE}(k, a, b, t, r, m),$$

where the procedure WEE defined on the facing page iterates through the elements of the w -bit words returned by $\text{chop}(k)$, applying f_a^r to the sum of the current word k_i and the previously computed hash value so far, finally returning the result obtained modulo m . This definition for variable-length and long (multiple-word) inputs is a consistent extension of the definition in equation (11.7) for short (single-word) inputs. For practical use, we recommend that a be a randomly chosen odd w -bit word, b be a randomly chosen w -bit word, and that $r = 4$.

Note that the wee hash function is really a hash function family, with individual hash functions determined by parameters a, b, t, r , and m . The (approximate) 5-independence of the wee hash function family for variable-length inputs can be argued based on the assumption that the 1-word wee hash function is a random oracle and on the security of the cipher-block-chaining message authentication code (CBC-MAC), as studied by Bellare et al. [42]. The case here is actually simpler than that studied in the literature, since if two messages have different lengths t and t' , then their “keys” are different: $a + 2t \neq a + 2t'$. We omit the details.

```

WEE( $k, a, b, t, r, m$ )
1   $u = \lceil t/w \rceil$ 
2   $\langle k_1, k_2, \dots, k_u \rangle = \text{chop}(k)$ 
3   $q = b$ 
4  for  $i = 1$  to  $u$ 
5       $q = f_{a+2t}^{(r)}(k_i + q)$ 
6  return  $q \bmod m$ 

```

This definition of a cryptographically inspired hash-function family is meant to be realistic, yet only illustrative, and many variations and improvements are possible. See the chapter notes for suggestions.

In summary, we see that when the memory system is hierarchical, it becomes advantageous to use linear probing (a special case of double hashing), since successive probes tend to stay in the same cache block. Furthermore, hash functions that can be implemented using only the computer's fast registers are exceptionally efficient, so they can be quite complex and even cryptographically inspired, providing the high degree of independence needed for linear probing to work most efficiently.

Exercises

★ 11.5-1

Complete the argument that for any odd positive integer a and any integer $r \geq 0$, the function $f_a^{(r)}$ is one-to-one. Use a proof by contradiction and make use of the fact that the function f_a works modulo 2^w .

★ 11.5-2

Argue that a random oracle is 5-independent.

★ 11.5-3

Consider what happens to the value $f_a^{(r)}(k)$ as we flip a single bit k_i of the input value k , for various values of r . Let $k = \sum_{i=0}^{w-1} k_i 2^i$ and $g_a(k) = \sum_{j=0}^{w-1} b_j 2^j$ define the bit values k_i in the input (with k_0 the least-significant bit) and the bit values b_j in $g_a(k) = (2k^2 + ak) \bmod 2^w$ (where $g_a(k)$ is the value that, when its halves are swapped, becomes $f_a(k)$). Suppose that flipping a single bit k_i of the input k may cause any bit b_j of $g_a(k)$ to flip, for $j \geq i$. What is the least value of r for which flipping the value of any single bit k_i may cause *any* bit of the output $f_a^{(r)}(k)$ to flip? Explain.

Problems
11-1 Longest-probe bound for hashing

Suppose you are using an open-addressed hash table of size m to store $n \leq m/2$ items.

- a.* Assuming independent uniform permutation hashing, show that for $i = 1, 2, \dots, n$, the probability is at most 2^{-p} that the i th insertion requires strictly more than p probes.
- b.* Show that for $i = 1, 2, \dots, n$, the probability is $O(1/n^2)$ that the i th insertion requires more than $2 \lg n$ probes.

Let the random variable X_i denote the number of probes required by the i th insertion. You have shown in part (b) that $\Pr\{X_i > 2 \lg n\} = O(1/n^2)$. Let the random variable $X = \max\{X_i : 1 \leq i \leq n\}$ denote the maximum number of probes required by any of the n insertions.

- c.* Show that $\Pr\{X > 2 \lg n\} = O(1/n)$.
- d.* Show that the expected length $E[X]$ of the longest probe sequence is $O(\lg n)$.

11-2 Searching a static set

You are asked to implement a searchable set of n elements in which the keys are numbers. The set is static (no INSERT or DELETE operations), and the only operation required is SEARCH. You are given an arbitrary amount of time to preprocess the n elements so that SEARCH operations run quickly.

- a.* Show how to implement SEARCH in $O(\lg n)$ worst-case time using no extra storage beyond what is needed to store the elements of the set themselves.
- b.* Consider implementing the set by open-address hashing on m slots, and assume independent uniform permutation hashing. What is the minimum amount of extra storage $m - n$ required to make the average performance of an unsuccessful SEARCH operation be at least as good as the bound in part (a)? Your answer should be an asymptotic bound on $m - n$ in terms of n .

11-3 Slot-size bound for chaining

Given a hash table with n slots, with collisions resolved by chaining, suppose that n keys are inserted into the table. Each key is equally likely to be hashed to each slot. Let M be the maximum number of keys in any slot after all the keys have